

Luana Rodrigues Borges
12311BCC028

Renan Henrique de Oliveira Silveira
12311BCC036

DESAFIO 1

Inteligência **computacional**

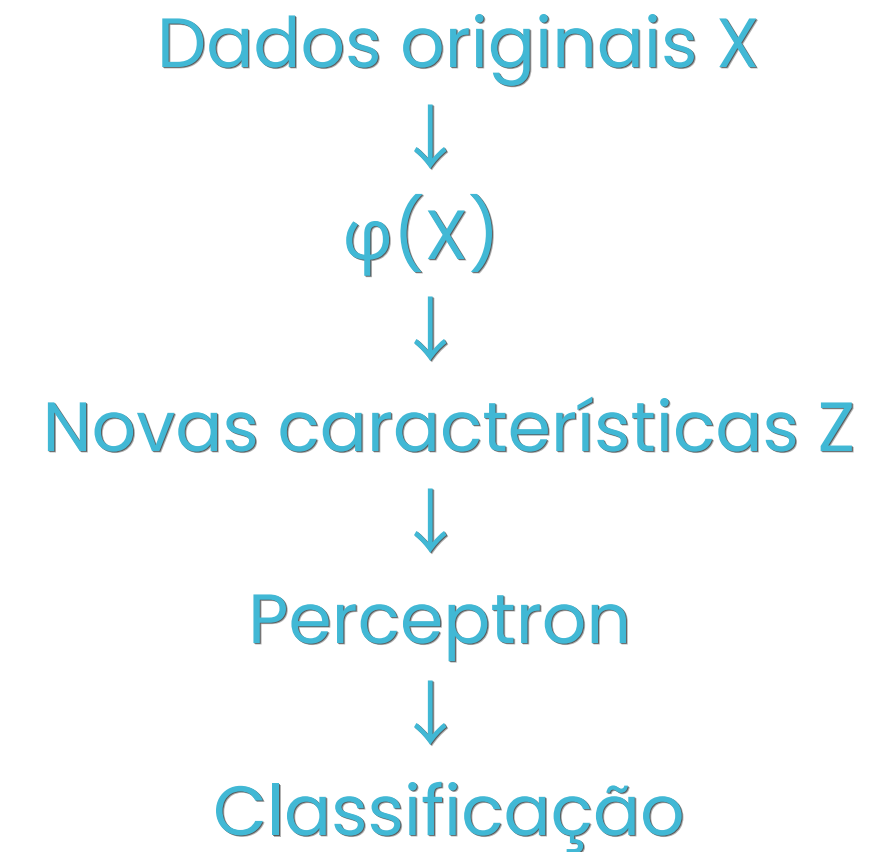


Mapa de Características

Objetivo: Melhorar a capacidade de um Perceptron linear de resolver problemas não linearmente separáveis.

A solução consiste em transformar os dados antes de entregá-los ao Perceptron, por meio de uma função phi, que adiciona mais características aos dados de entrada.

- Limite máximo: 64 características.



Padronização

POR QUE FAZER ISSO?

Porque as características passam a ficar em escalas comparáveis.

```
def fit(self, X: torch.Tensor) -> None:
    d = X.shape[1] # num features
    self.mu = X.mean(0) # media
    self.sd = X.std(0) + 1e-8 # desvio padrao
    Xs = (X - self.mu) / self.sd
```

- **mu** → média de cada característica
- **sd** → desvio padrão
- **1e-8** → evita divisão por zero
- **Xs** → dados padronizados

Características quadráticas

Se tivermos:

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2]$$

```
partes = [Xs, Xs ** 2]
```

passamos a ter:

$$\varphi(\mathbf{X}) = [\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_1^2, \mathbf{x}_2^2]$$

Exemplo:

$$\mathbf{x}_1^2 + \mathbf{x}_2^2$$

Isso permite representar relações que dependem do quadrado das variáveis.

é relacionado à distância até a origem → muito útil para **padrões circulares**.

```
# pares (i, j) só entram se couberem no orçamento
n_pares = d * (d - 1) // 2 # qnt pares de colunas
# se d >= 2 e n_pares <= 20, gera combinações de pares (i,j)
self.pair_idx = torch.combinations(torch.arange(d), r=2) if d >= 2 and n_pares <= 20 else None
# numero de linhas de pair_idx = quantidade de pares de colunas (novas features)
pair_dim = self.pair_idx.shape[0] if self.pair_idx is not None else 0
```

Para duas características:

$[x_1, x_2]$

Adicionamos:

$x_1 \cdot x_2$

```
if self.pair_idx is not None:
    # separa todos pares ([0,1] -> [0], [1])
    i, j = self.pair_idx[:, 0], self.pair_idx[:, 1]
    # calcula produto entre cada par de elementos xi * xj
    partes.append(Xs[:, i] * Xs[:, j])
```

Então:

$[x_1, x_2] \rightarrow [x_1, x_2, x_1^2, x_2^2, x_1x_2]$

Porque algumas classes dependem da interação entre **duas variáveis**

Exemplo: XOR

Interações entre características

Características de Fourier

Para duas características:

$[x_1, x_2]$

Multiplica por pesos aleatórios W:

$$\text{proj}_k = x_1 \cdot w_{1k} + x_2 \cdot w_{2k}$$

Aplica cos() e sin() em cada projeção:

$\cos(\text{proj}_1), \sin(\text{proj}_1), \cos(\text{proj}_2), \sin(\text{proj}_2)$

Então:

$[x_1, x_2] \rightarrow [x_1, x_2, x_1^2, x_2^2, x_1x_2, \cos(\text{proj}_1), \sin(\text{proj}_1), \cos(\text{proj}_2), \sin(\text{proj}_2), \dots]$

```
# gera pesos aleatorios para fourier
self.W = torch.randn(d, self.n_freq, generator=g) / escala_por_freq
```

```
def phi(self, X: torch.Tensor) -> torch.Tensor:
    # X com mesmos valores de media e desvio padrao do fit
    Xs = (X - self.mu) / self.sd
    # features
    partes = [Xs, Xs ** 2]
```

```
# multiplica Xs @ W
proj = Xs.double() @ self.W.double()
# calcula cos e sen (fourier)
partes += [torch.cos(proj).float(), torch.sin(proj).float()]
```

Para padrões mais complexos, como a espiral, somente os termos quadráticos não são suficientes.

Por isso eu também utilizamos Random Fourier Features, que fazem projeções aleatórias dos dados e aplicam funções periódicas, seno e cosseno.

Múltiplas escalas

```
amostra = Xs[:min(len(Xs), 300)] # pega 300 primeiras amostras de Xs
dists = torch.pdist(amostra) # distancia entre todos pares da amostra

sigma_base = dists.median().item() if dists.numel() else 1.0 #
sigma_base = sigma_base if sigma_base > 1e-8 else 1.0

log_min, log_max = math.log(0.12 * sigma_base), math.log(6.0 * sigma_base)

escala_por_freq = torch.exp(log_min + torch.rand(self.n_freq, generator=g) * (log_max - log_min))

# gera pesos aleatorios para fourier
self.W = torch.randn(d, self.n_freq, generator=g) / escala_por_freq
```

escala pequena → frequência maior → padrões mais detalhados

escala grande → frequência menor → padrões mais suaves

1. Encontramos as **distâncias** entre os pontos dos dados
2. Calculamos o **sigma_base**, que é a **mediana** dessas distâncias e representa uma **escala típica** do conjunto de dados
3. **Log min e max** definem os limites das escalas que serão exploradas
4. Cada frequência recebe uma escala diferente
5. **W** recebe as frequências usadas nas projeções de Fourier

Resultados

tarefa	baseline	referência	você	s_t
xor	0.606	0.876	0.864	0.96
duas_luas	0.892	0.988	0.983	0.95
circulos	0.644	1.000	1.000	1.00
espiral	0.633	0.912	0.923	1.04
esfera_10d	0.508	0.872	0.960	1.24

ESCORE S = 101.0 (0 = baseline, 100 = referência, até 125 com bônus)

Xor: o termo de interação ($x_1 \cdot x_2$) é a característica principal, já que a fronteira é definida pelo sinal do produto entre as variáveis.

Duas luas: as características de Fourier (cos/sin) aproximam a curva suave que separa as duas luas.

Circulos: os termos quadráticos ($x_1^2 + x_2^2$) resolvem bem esse padrão, porque a fronteira é $\text{raio}^2 > \text{limiar}$.

Espiral: as características de Fourier em múltiplas escalas ajudam a capturar as voltas do espiral.

Esfera 10d: os termos quadráticos generalizam bem pra alta dimensão, já que a fronteira é soma dos quadrados $> \text{limiar}$.

NOSSA SOLUÇÃO COMBINA:

Padronização + X e X^2 + $X_i \cdot X_j$ + RFF + múltiplas escalas



Melhor classificação do Perceptron

Seguindo os critérios:

- O Perceptron não foi modificado.
- A melhoria veio da representação dos dados.
- Diferentes tipos de características ajudam em diferentes padrões.
- A solução respeita o limite de 64 dimensões.
- A transformação é determinística pois usamos um seed fixa.

Conclusão



Obrigado.